

THEGAPBRIEF · SIDE REPORT · NOVEMBER 2, 2026

Running a Bot Team on Less: Conserving Grok Bot Usage

Most of Jacob's Grok Bot usage is chat work, not routines. Better cron hygiene helps a little. Changing how the Kapsoul bots get briefed, how long their threads run, and how much they talk to each other saves far more.

Assigned by Jacob Peterson · Research. Checked Nov 2, 2026. Points tagged **[Jacob's data]** come from Jacob's own usage report for Oct 3 – Nov 2, 2026, given as shares of all Grok Bot usage. Points tagged **[Best practice]** are general guidance. This report gives no projected savings or run counts beyond the report's own numbers. Nothing was changed; this is a report only.

SUMMARY Bottom line

1 The routines aren't the problem. The chat is.

[Jacob's data] About **85–90%** of usage is interactive or delegated chat work, not routines. Kapsoul's three bots (Sentinel 17%, Atlas 13%, Forge 10%) make up about **40%** of the total, nearly all of it outside routines. Even the busiest routine, ShipSonar's stuck-order watch (about 336 runs a week), is only about 1%.

Top 3: (1) Put the Kapsoul bots on tight briefs and fresh threads. (2) Cut bot-to-bot chatter: status pings, wrap-ups and relays. (3) Move deterministic checks such as stuck orders, sensitive paths and deadline reminders to plain scripts, webhooks or GitHub Actions. The routine fixes (lower cadence, waking hours only, triggers) are easy, but they're the smaller wins.

Source: Jacob's usage report, Nov 2, 2026 (last 30 days).

0 Where the usage went (Oct 3 – Nov 2)

Bot / group	Share	Notes from the report
Sentinel (Kapsoul QA)	17%	All outside routines
Atlas (Kapsoul backend)	13%	Almost all outside routines
Forge (Kapsoul web)	10%	—
Hull (ShipSonar scanner)	9%	Several status routines paused
Keeper (social)	7%	5% outside routines. IG staggered follows 1% (~42 runs/wk). X lead scout, Higgsfield Discord reply check and IG daily run each ~14/wk. Discord copycat watch ~7/wk. Other routines 1%
Other bots	44%	41% outside routines. Kapsoul safety deadline reminders 1% (~56/wk). ShipSonar stuck-order watch 1% (~336/wk, every 30 min). ShipSonar 12-hour sensitive-path check ~14/wk. TheGapBrief weekly <1%

0.1 Where things stand this week

[Jacob's data] Plan: Cursor Ultra. The weekly usage limit is **74.6% used** and resets **Thu Nov 5, 12:10 PT**. On-demand is **$20.47 of the $30** monthly limit, which covers Grok Bot and Cursor together, and resets Sun Nov 8. On-demand was 1% of Grok Bot usage in this period.

[Best practice] Cursor says its included pool is shared by everything on the subscription: the editor's Agent, the CLI, Cloud Agents, Grok Bot and the Grok apps. Heavy Cursor coding and heavy bot chat use the same budget. Unused included usage doesn't roll over.

Cursor: Models & Pricing · Cursor: Usage and limits

0.2 What the numbers say

1) The work is chat-heavy. Routines across all bots are a small minority. 2) Kapsoul is the hot spot, and its QA bot leads. 3) Run counts and usage don't line up: 336 runs a week of the stuck-order watch is about 1%, the same as Keeper's 42-a-week follows. A cheap run done often isn't the problem. Long, open-ended conversations are.

Interpretation of Jacob's usage report.

---

1 Ranked recommendations

1.1 Tight briefs and fresh threads for Sentinel, Atlas and Forge *[Jacob's data + Best practice]*

What changes: give each Kapsoul task one goal, the files or areas it covers, a "done when" line and an output format, such as "Return a pass/fail list, max 10 lines." Start a new thread for each task instead of continuing one long thread for days. Paste a 5-line handoff summary in place of the old history.

Why: Cursor counts context as input tokens. On Grok 4.7, input past 256k tokens is billed at 2× standard rates. A long thread re-sends its history every turn, so later turns cost more than earlier ones. Open-ended QA ("test everything") leads to wide exploration. Sentinel alone is 17%.

Tradeoff: it takes a minute of setup per task, and a new thread loses whatever history you didn't carry over. Keep a short "Kapsoul state" note the bots can read.

Cursor: Models & Pricing (long-context rates) · Cursor: larger context increases tokens

1.2 Cut chat between bots *[Jacob's data + Best practice]*

What changes: no status pings, "acknowledged" replies or daily wrap-ups between bots. Bots message each other only to hand off work or flag a blocker. Results go into one shared file or page that people read, instead of being sent around. Don't use a PM bot that relays between bots and repeats their output.

Why: every message to a bot wakes it, which starts a full turn with context loaded. 41% of "other bots" usage happens outside routines, which is where relay chatter shows up. Grok Bot's own usage guidance says the same: messaging another bot wakes it and uses usage.

Tradeoff: less real-time awareness, so you read the shared status file instead of getting updates pushed to you.

Jacob's usage report; this bot's usage-review guidance.

1.3 Move deterministic checks out of Grok Bot *[Jacob's data + Best practice]*

What changes: replace checks that need no judgment with a script on cron, a GitHub Actions `schedule` workflow, or a Supabase/Vercel cron. That covers the stuck-order watch (a query plus a threshold), the 12-hour sensitive-path check (a diff or grep), and the Kapsoul safety deadline reminders (date math). The script calls a bot only when something is actually wrong. This was already done for PromptPatrol face watch.

Why: a script costs no Grok Bot usage on the runs where everything is fine. The bot is only needed to interpret a problem.

Tradeoff: someone has to maintain a small script and its secrets, and an unexpected edge case won't get judged by a bot.

GitHub Docs: schedule event · Vercel Cron Jobs · Supabase Cron

1.4 Switch from schedules to triggers *[Jacob's data + Best practice]*

What changes: instead of polling every 30 minutes for stuck orders, react to events: a Stripe webhook (such as a failed or delayed payment), a GitHub push for sensitive paths, or an email or Slack message.

Why: a trigger only runs when something happens. The stuck-order watch runs about 336 times a week, and most of those runs presumably find nothing.

Tradeoff: short setup, the app has to be connected, and a missing event (one that never fires) needs a backup, such as one daily sweep.

Stripe: Webhooks · GitHub: Webhooks

1.5 Lower cadence, waking hours only, and merge duplicates *[Jacob's data]*

What changes: run Kapsoul deadline reminders (~56/wk) a few times on weekdays only. Combine Keeper's X lead scout, Higgsfield Discord reply check and Discord copycat watch into one "social sweep" twice a day in waking hours. Consider merging the IG staggered follows (~42/wk) with the IG daily run. Delete, rather than keep, any of Hull's paused status routines that are no longer needed.

Why: nobody acts on results at 3 AM, and one merged run loads context once instead of three times.

Tradeoff: results are less fresh, and overnight issues wait until morning. These routines are each about 1%, so this is cleanup, not the main fix.

Jacob's usage report.

1.6 Low effort for lookups, high effort only for hard problems *[Best practice]*

What changes: ask for a quick answer on simple facts, status reads and formatting. Save deep reasoning and multi-source research for architecture, debugging and reports.

Why: Cursor says model choice affects how fast included usage runs down, and its price table shows "Fast" variants at about 2× standard rates.

Tradeoff: a quick answer can miss a subtlety, so escalate when one does.

Cursor: Usage and limits · Cursor: Models & Pricing

1.7 Cache research and reuse it *[Best practice]*

What changes: save research results (pricing tables, API limits, competitor notes) to files in the repo or store with a "checked on" date. Bots read the file before searching the web again. TheGapBrief reports already work as a cache.

Why: re-researching the same facts costs search and page reads every time. Reading one short file costs much less.

Tradeoff: cached facts go stale. Set a re-check date, such as 30 days.

General practice.

1.8 Use the browser and desktop only as a last resort *[Best practice]*

What changes: prefer a connector or API (Zernio, GitHub, Supabase, Stripe) over driving a browser. When browser work is needed, give the exact URL and the exact thing to read.

Why: screenshot-and-click loops take many steps and carry a lot of context per step. An API call is one step.

Tradeoff: some sites have no API.

General practice.

1.9 Batch asks, and merge overlapping bots *[Jacob's data + Best practice]*

What changes: send one message with five asks instead of five messages. Consider merging the Kapsoul bots into fewer bots, or have Forge and Atlas share a single "Kapsoul state" file so each doesn't rebuild the same picture.

Why: each new turn or new bot loads its context again.

Tradeoff: a merged bot has a bigger, more general role and can lose focus. Merge only where the work overlaps.

Jacob's usage report (Kapsoul ≈ 40%).

1.10 Watch the on-demand cap and do a weekly review *[Jacob's data + Best practice]*

What changes: on-demand is at $20.47 of $30 with the weekly limit 74.6% used, so go easy until Thu Nov 5. Keep the $30 cap, or lower it, so usage stops at a fixed point. The app already warns as you near the limit, so you don't need a routine to watch usage (that check would use usage too). Every Monday, spend 5 minutes on the shares by bot and routine: check the top 3 bots, pause one routine, and fix one brief.

Why: nothing stops automatically at a percentage of included usage. The spend limit is the hard stop. Cursor says on-demand is billed in arrears at API rates.

Tradeoff: work stops at the cap until the reset.

Cursor: Usage-based charges · Cursor: Legacy plans, on-demand

2 Ranked at a glance

Rank	Lever	Targets	Effort
1	Tight briefs + fresh threads	Kapsoul bots (~40%)	Low

2	Cut chat between bots	Other bots' 41% outside routines	Low
3	Move checks to scripts/cron/Actions	Stuck-order, sensitive-path, deadline reminders	Medium
4	Triggers instead of polling	Stuck-order watch (Stripe webhook)	Medium
5	Cadence, waking hours, merges	Keeper + reminders	Low
6–10	Effort level, caching, browser, batching, review	Everything	Low

No savings are projected; check the shares by bot and routine one week after each change.

TheGapBrief Side Reports are published by Open Mind Innovations LLC. Researched November 2, 2026. Usage figures are from Jacob's own usage report; outside facts are as shown by the linked sources on that date.